

Phase 3 Report

Kubernetes Workload Analyzer — Go backend skeleton, verified against the live cluster

Phase status	DONE (2026-09-25)
Report date	2026-09-26
Repository	github.com/ISHANT57/kubernetes-workload-analyzer
State of main	65cb805 , merged as d19c5d3 (PR #7)
Cluster	kind workload-analyzer , Kubernetes v1.37.0, single node
Written for	the project owner, as a study and reference document

Evidence labels used throughout. **MEASURED** a command was run on this machine and the number is its output · **VERIFIED** checked against upstream documentation, source or a registry · **ESTIMATED** a projection, not measured · **UNVERIFIED** not yet tested; a test is named.

Contents

1. What Phase 3 was for
2. Package layout
3. The analysis loop design
4. Kubernetes and Prometheus clients (D-007 scope)
5. HTTP surface
6. Unit tests (fakes, no live cluster)
7. Live verification against the real cluster
8. Graceful shutdown
9. Git and GitHub workflow
10. Current state, operating and rollback
11. Open items and next phase

1. What Phase 3 was for

Phases 1-2 built the data layer (cluster, monitoring stack, verified metrics) and the demo fixtures. Phase 3 built the backend's **skeleton**: configuration, the Prometheus and Kubernetes clients, the analysis loop, and the operational HTTP surface (`/healthz//readyz//metrics//api/runs/latest`). Deliberately out of scope for Phase 3: the rule engine, evidence builder, cost model and `/api/findings` — those are Phase 4. The done-condition (`PHASES.md`): "Runs against the cluster; degrades cleanly with Prometheus down."

2. Package layout

Go 1.27 module under `backend/`, chosen and recorded as D-001. Every external call sits behind a small interface so the whole module except `cmd/*` is testable with fakes, no live cluster required (`backend/README.md`).

<code>cmd/analyzer/main.go</code>	wiring only: config -> clients -> runner -> http server
<code>internal/config/</code>	env-var configuration, validated
<code>internal/logging/</code>	structured JSON logging (stdlib log/slog)
<code>internal/model/</code>	dependency-free domain types (AnalysisRun, WorkloadRef, ...)
<code>internal/promclient/</code>	Prometheus client behind an interface (instant + range queries)
<code>internal/k8sclient/</code>	Kubernetes client behind an interface, scoped to D-007
<code>internal/runner/</code>	the analysis loop: timer, failure handling, in-memory snapshot
<code>internal/httpserver/</code>	<code>/healthz /readyz /metrics /api/runs/latest</code>
<code>internal/metrics/</code>	the analyzer's own Prometheus metrics (self-observability)

3. The analysis loop design

An in-process ticker runs on `ANALYSIS_INTERVAL`, lists workloads from the Kubernetes API, and keeps the latest result as an in-memory **snapshot** (`internal/runner`) — no queue, no worker process, no database (`docs/architecture.md`, "Shape: modular monolith"). Each cycle produces a `model.AnalysisRun` with a status of `complete`, `partial` or `failed`. On a source failure, the runner keeps the **previous** snapshot rather than blanking it — the sticky-snapshot pattern used consistently in every later phase too (findings in Phase 4, the dashboard's polling hook in Phase 5).

4. Kubernetes and Prometheus clients (D-007 scope)

`internal/k8sclient` lists namespaces, nodes, deployments, replicaset, statefulsets, daemonsets, HPAs and events only — the read-only ClusterRole decided in D-007. No `pods`, `secrets` or `configmaps` access exists in the client at all, not just in the RBAC that constrains it: a coding mistake could not accidentally list pods even before the in-cluster RBAC is applied (that live RBAC verification is Phase 7). `internal/promclient` wraps `client_golang/api/prometheus/v1` for instant and range queries, with per-query timeouts.

5. HTTP surface

Endpoint	Behaviour
/healthz	process liveness — always ok while the process is up
/readyz	200 only when the last analysis run was complete ; 503 otherwise
/metrics	the analyzer's own Prometheus metrics (internal/metrics): run count by status, run duration histogram, query error count, workloads-seen gauge
/api/runs/latest	the current in-memory AnalysisRun snapshot as JSON

6. Unit tests (fakes, no live cluster)

```
go vet ./...  
gofmt -l .    # prints nothing  
go test ./...  # all packages pass
```

Notable tests, both later also live-verified against the real cluster (§7): `TestRunOnce_PrometheusDown` and `TestLatest_WorkloadsSeenSurvivesAFailedRun` in `internal/runner` — the "degrades cleanly" done-condition, exercised with a fake Prometheus client that returns an error.

7. Live verification against the real cluster

Run with the real kind cluster and a real Prometheus port-forward, not just against fakes:

Results (2026-09-25):

- Correctly counted **16 deployments** from the live Kubernetes API on the first run.
- Prometheus access was killed mid-run and restored live: `/readyz` correctly went **503** during the outage and recovered to **200** once Prometheus was reachable again.
- `workloads_seen` stayed at **16** throughout the outage — carried forward from the last successful Kubernetes listing, never dropped to 0 by a Prometheus failure (Kubernetes and Prometheus are independent sources; one failing does not blank the other's last-good data).
- `/healthz` stayed unaffected the whole time (process liveness, not dependency health).
- Graceful `SIGTERM` shutdown confirmed: the process drains and exits cleanly, not killed mid-request.

8. Graceful shutdown

`cmd/analyzer/main.go` listens for `SIGTERM/SIGINT` and shuts the HTTP server down with a bounded context, rather than the process dying mid-request — the behaviour a Kubernetes `Deployment` rollout or `kubectl delete pod` relies on to avoid a dropped request. Verified locally (§7); re-verified in-cluster during Phase 7's rollout.

9. Git and GitHub workflow

Branch `phase-3` → commit `65cb805` ("feat(phase-3): Go backend skeleton, verified against the live cluster") → PR #7 → merged as `d19c5d3` → branch deleted locally and on GitHub.

`deploy/rbac/analyzer.yaml` was written in this phase but deliberately **not yet applied** to the cluster — in-cluster deployment is Phase 7; Phase 3 development still runs on the host against the admin kubeconfig.

10. Current state, operating and rollback

```
kubectll -n monitoring port-forward svc/kps-kube-prometheus-stack-prometheus 9090:9090
cd backend && CLUSTER_ID=workload-analyzer KUBE_CONTEXT=kind-workload-analyzer go run ./cmd/analyzer
curl localhost:8080/healthz
curl localhost:8080/readyz
curl localhost:8080/api/runs/latest
```

Rollback: the skeleton has no in-cluster footprint yet (host-only in this phase), so rollback is simply not running the process / reverting the merge commit.

11. Open items and next phase

- No rule engine, evidence builder, cost model or `/api/findings` yet — Phase 4.
- `deploy/rbac/analyzer.yaml` written but not applied — Phase 7.
- No dashboard yet — the API has no consumer other than `curl` until Phase 5.